NLP Fundamentals Series

Text Preprocessing in NLP

Tokenization · Lemmatization · Stopwords · NLTK · spaCy

Mosharof Hossain

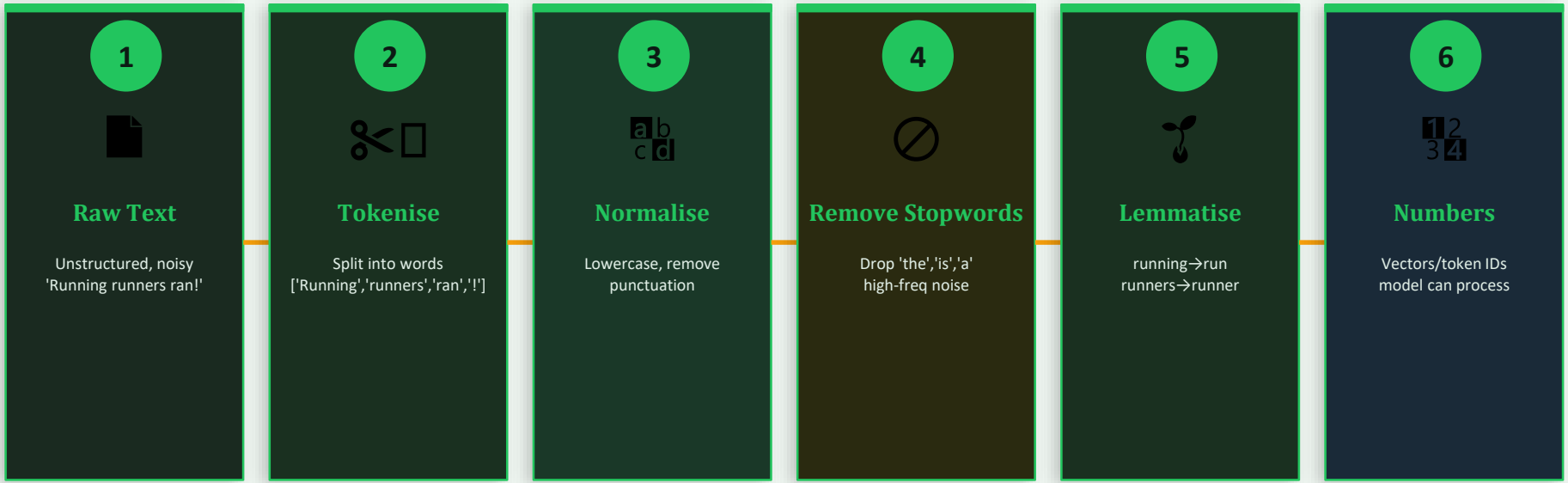
AI Engineer · Brain Station 23

Why Text Preprocessing?

Raw text is messy — machines need structured numbers

Machines cannot read words. Before any NLP model — classification, chatbot, search — can work, raw text must be **cleaned, broken into units, and converted to numbers.** That transformation pipeline is called text preprocessing.

The NLP Preprocessing Pipeline



- Better accuracy
- Faster training
- Generalization
- Lower cost

Tokenization — Breaking Text into Units

Word · Sentence · Subword tokenization + NLTK & spaCy demos

Tokenization is splitting raw text into smaller meaningful units called tokens. These can be **words, sentences, subwords, or characters** — depending on the task and the model.

Word Tokenization

```
Input:  "NLP is amazing!"

Output: ['NLP', 'is', 'amazing', '!']
```

Most common. Good for English. Splits on whitespace & punctuation.

⚠ Problem: 'run','running','ran' are 3 different tokens — model sees them as unrelated.

```
word_tokenize("NLP is amazing!")

→ ['NLP', 'is', 'amazing', '!']
```

Sentence Tokenization

```
Input:  "I love NLP. It is powerful!"

Output: ['I love NLP.', 'It is
powerful!']
```

Split documents into sentences — useful for summarization, QA, NLI tasks.

⚠ Tricky with abbreviations: 'Dr. Smith went home.' — period after Dr. is not end of sentence.

```
sent_tokenize("I love NLP. It...")

→ ['I love NLP.', 'It is powerful!']
```

⚡ Subword Tokenization

```
Input:  "playing"

Output: ['play', '##ing']  (BERT
WordPiece)
```

Used by BERT, GPT — handles unknown words by splitting into known subwords.

⚠ Less readable for humans, but machines love it — zero unknown tokens.

```
tokenizer("playing")  # BERT

→ ['play', '##ing']  (BERT WordPiece)
```

Stopwords — Filtering the Noise

Words that appear everywhere but carry almost no meaning

Stopwords are extremely common words — 'the', 'is', 'in', 'a', 'and' — that appear in almost every sentence but carry **little semantic value.** Removing them reduces noise and speeds up model training.

Before vs After Stopword Removal:

Original sentence:

The quick brown fox is running across the beautiful green field in summer

↓ Remove stopwords (red = removed)

After removal:

quick brown fox running beautiful green field summer

NLTK — Stopword Removal

179 English stopwords

```
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

stop_words = set(stopwords.words('english'))
tokens = word_tokenize(text)

filtered = [w for w in tokens
```

stopwords.words('english') — also supports 'french','german','arabic'...

spaCy — Stopword Removal

326 English stopwords

```
import spacy
nlp = spacy.load('en_core_web_sm')

doc = nlp(text)

filtered = [token.text
    for token in doc
```

token.is_stop is True for any stopword. Context-aware — more complete list.

Stemming — Chopping Words to Their Root

A fast, rule-based way to normalise words — no dictionary needed

Stemming is the process of reducing a word to its **base or root form** by chopping off suffixes using simple rules — **no grammar knowledge or dictionary needed.** Fast but imprecise.

How Stemming Works — Rule-Based Suffix Stripping:

Word	Rule Applied	Stem Result	Valid word?
running	*remove -ing*	run	✓✓ Yes
studies	*ies→y, remove -y*	studi	✗ No
happily	*remove -ily*	happi	✗ No
connection	*remove -ion*	connect	✓✓ Yes
generous	*remove -ous*	gener	✗ No
walking	*remove -ing*	walk	✓✓ Yes
better	*no rule matches*	better	✗ No
flies	*ies→i*	fli	✗ No

⚠ Stemming breaks words into non-real 'roots' (studi, happi, gener) — fast but loses meaning. Use Lemmatization when accuracy matters.

Three Stemming Algorithms — Porter · Snowball ·Lancaster

Aggressive vs balanced vs conservative stemming

🌿 Porter Stemmer

Martin Porter, Cambridge (1980)

Speed: Fast | Aggression: Moderate

5-phase rule system. Removes common English suffixes in sequence.
Phase 1: SSES→SS, IES→I, SS→SS, S→
Phase 2: ATIONAL→ATE, TIONAL→TION...

✓ Most widely used

✓ Balanced accuracy

✓ Well-tested since 1980

✗ English only

💡 Default choice for English NLP

```
from nltk.stem import PorterStemmer
porter = PorterStemmer()
porter.stem('running')     # → 'run'
porter.stem('studies')     # → 'studi'
porter.stem('generously') # → 'generous'
```

❄□ Snowball Stemmer

Martin Porter — improved version (2001)

Speed: Fast | Aggression: Moderate-Low

Also called 'Porter2'. Improved algorithm over original Porter. Less aggressive, more accurate.
Supports 13+ languages (French, German, Arabic...).

✓ Multi-language support

✓ More accurate than Porter

✓ Active maintenance

✗ Still rule-based

💡 When you need multilingual or improved English stemming

```
from nltk.stem import SnowballStemmer
snow = SnowballStemmer('english')
snow.stem('running')      # → 'run'
snow.stem('generously')   # → 'generous'
# Other langs:
SnowballStemmer('french').stem('mangeaient')
```

⚡ Lancaster Stemmer

Paice & Husk, Lancaster University (1990)

Speed: Fastest | Aggression: Very High

Iterative algorithm — keeps applying rules until no more apply.
Very aggressive: strips more than Porter or Snowball.
Result is often not a real word.

✓ Smallest stems (most compression)

✓ Very fast

✓ Good for clustering

✗ Over-stems heavily

💡 When you want maximum compression, accuracy secondary

✗ Worse accuracy

```
from nltk.stem import LancasterStemmer
lanc = LancasterStemmer()
lanc.stem('running')      # → 'run'
lanc.stem('generously') # → 'gen'
lanc.stem('maximum')     # → 'maxim'
```

Lemmatization — Words to Their Base Form

Reduces inflected words to dictionary root using grammar context

Lemmatization converts words to their base dictionary form (lemma) using vocabulary and morphological analysis. Unlike stemming, it considers **the word's part of speech** — so 'better' → 'good' (not 'better').

Stemming vs Lemmatization — Key Difference

Original	Stemmed (Porter)	Lemma	POS
running	run	**run**	verb
studies	studi	**study**	noun/verb
better	better	**good**	adjective
went	went	**go**	verb
corpora	corpora	**corpus**	noun
wolves	wolv	**wolf**	noun

NLTK WordNet Lemmatizer:

spaCy Lemmatizer (POS-aware):

```
import spacy

nlp =spacy.'en_core_web_sm'

text='The wolves ran fast'
doc = nlp(text)

# Each token has .lemma_ attribute

for token doc:

(   print

        token.text,

        '→',

        token.lemma_,
```

NLTK vs spaCy — Choosing the Right Library

Both do preprocessing — different philosophy and use cases

Feature	NLTK	spaCy
Philosophy	Educational, modular, explicit	Industrial-strength, opinionated
Speed	Slower (pure Python)	Very fast (Cython, Rust-backed)
Stemming	✓ Porter, Snowball, Lancaster	✗ Not supported (use lemma instead)
Lemmatization	✓ WordNet (POS required manually)	✓ Context-aware, POS auto-detected
Tokenization	word_tokenize, sent_tokenize	nlp(text) → Doc object
Stopwords	179 words (English)	326 words (English), more accurate
Pipeline	Manual: call each function separately	Automatic: one nlp() call does all
Best for	Learning NLP, research, custom rules	Production NLP, speed, accuracy

Rule of thumb: Use NLTK for learning and research. Use spaCy for any production system or large dataset.

HuggingFace Tokenizers — Modern Subword Tokenization

BPE · WordPiece · SentencePiece · AutoTokenizer

HuggingFace tokenizers (from the 🤗 **Tokenizers** library) use **subword algorithms** to handle any word — even out-of-vocabulary ones — by splitting into known subword pieces.

BPE

Byte-Pair Encoding

Model: GPT-2, RoBERTa, LLaMA

Start with characters. Repeatedly merge most frequent adjacent pairs into new tokens.

```
"playing" → ["play","ing"]
```

WordPiece

WordPiece Algorithm

Model: BERT, DistilBERT

Similar to BPE but maximises likelihood of training data. Uses ## prefix for non-initial subwords.

```
"playing" → ["play","##ing"]
```

SentencePiece

SentencePiece (Unigram)

Model: T5, mBART, LLaMA

Treats text as Unicode characters. Replaces spaces with ▁. Language-agnostic — works for Chinese/Arabic too.

```
"playing" → ["▁play","ing"]
```

AutoTokenizer — Use Any Model's Tokenizer in One Line:

```python
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained ("bert-base-uncased")
tokens    = tokenizer("The quick brown fox")
# Returns: input_ids, token_type_ids, attention_mask

print(tokenizer.convert_ids_to_tokens(tokens["input_ids"]))
# ['[CLS]', 'the', 'quick', 'brown', 'fox', '[SEP]']
```

Full Preprocessing Pipeline — All Steps Combined

NLTK · spaCy · HuggingFace — side by side

A real NLP pipeline combines all steps: clean text → tokenize → remove stopwords → lemmatize → encode. The output is **clean, normalised tokens ready for your model.**

Same text through the pipeline:

Raw Input: `"The running foxes are beautifully chasing small rabbits in the field."`

Tokenised:
```
['The','running','foxes','are','beautifully','chasing','small','rabbits','in','the','field','.'
]
```

Stop removed: `['running','foxes','beautifully','chasing','small','rabbits','field']`

Lemmatised: `['run','fox','beautifully','chase','small','rabbit','field']`

Implementations:

NLTK

```
# pip install nltk

import nltk
```

spaCy

```
# pip install spacy

# python -m spacy download en_core_web_sm
```

HuggingFace

```
# pip install transformers

from transformers import AutoTokenizer
```

Demo Plan + Session Summary

Run the notebooks live — see every step in action

Live Demo Steps

1. **NLTK Basics**
   word_tokenize, sent_tokenize, stopwords, WordNet lemmatizer
2. **spaCy Pipeline**
   nlp(text) → .is_stop, .lemma_, .pos_ in one pass
3. **NLTK vs spaCy Side-by-Side**
   Same text, compare token count, lemmas, stopword lists
4. **HuggingFace AutoTokenizer**
   BERT tokenizer → input_ids, attention_mask, subword splits
5. **Full Pipeline Function**
   Raw text → clean tokens → ready for ML. Compare all 3 outputs.
6. **Real Dataset Demo**
   Apply pipeline to IMDb reviews from HuggingFace datasets

What We Covered

Tokenization

Word, sentence, subword. Foundation of every NLP pipeline.

Stopwords

Remove noise. NLTK 179 words. spaCy 326 context-aware.

Lemmatization

Base form via POS. Better than stemming. spaCy auto-detects.

NLTK

Educational, modular, 50+ resources. Best for learning.

spaCy

Production speed. One pass does all. Context-aware lemma.

HuggingFace

AutoTokenizer → BPE/WordPiece. Plug into any model.

Next: Word Embeddings (Word2Vec, GloVe, FastText) — turning clean tokens into dense vectors for ML models